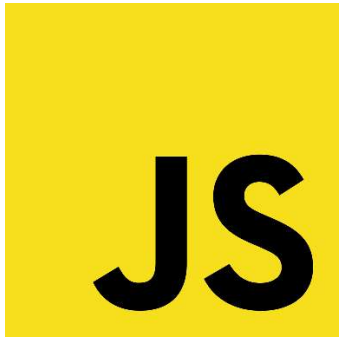
Short recap day #1

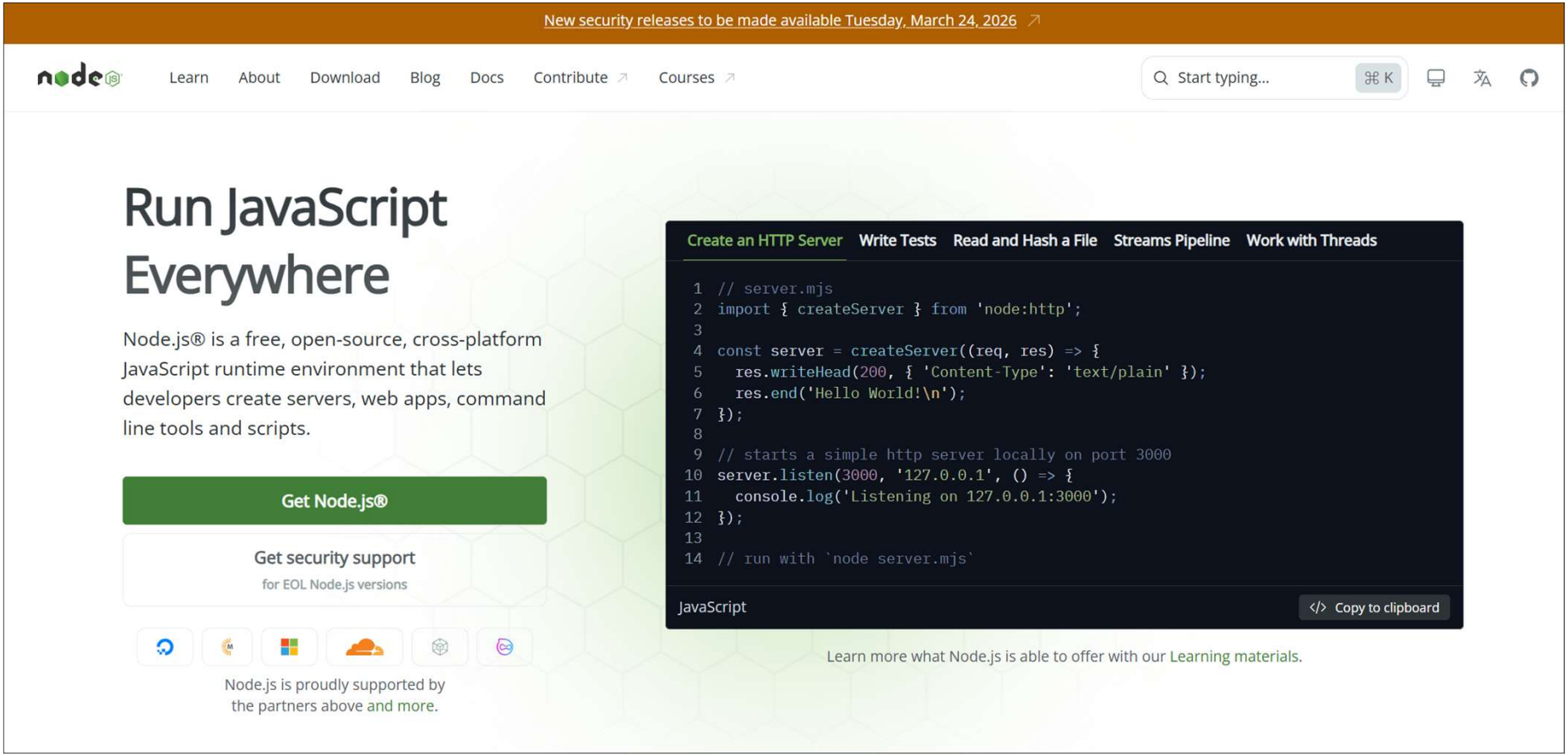
Peter Kassenaar – info@kassenaar.com

Front-end : one stack

- HTML, CSS, JavaScript
- Huge landscape – lots modules in one stack

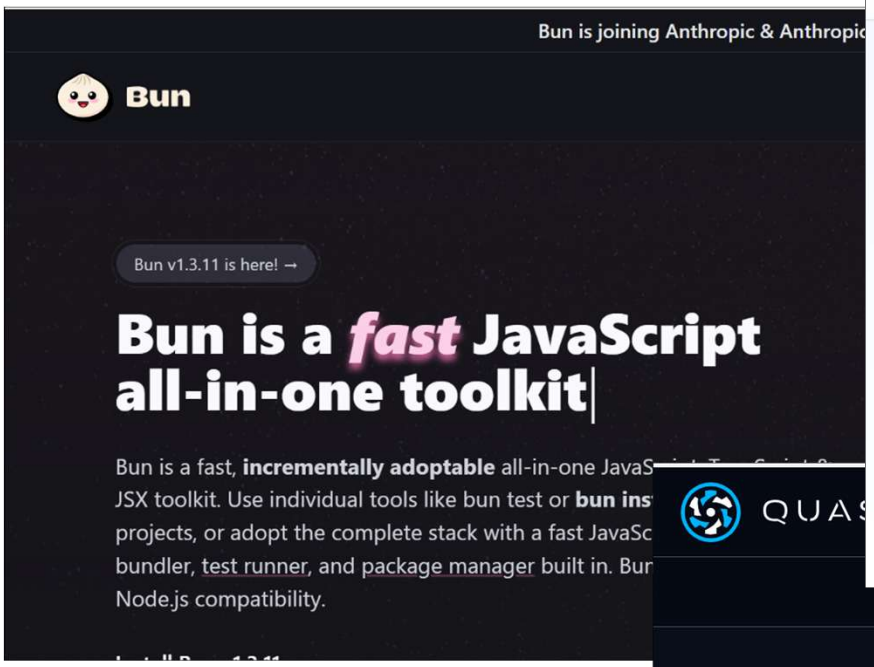


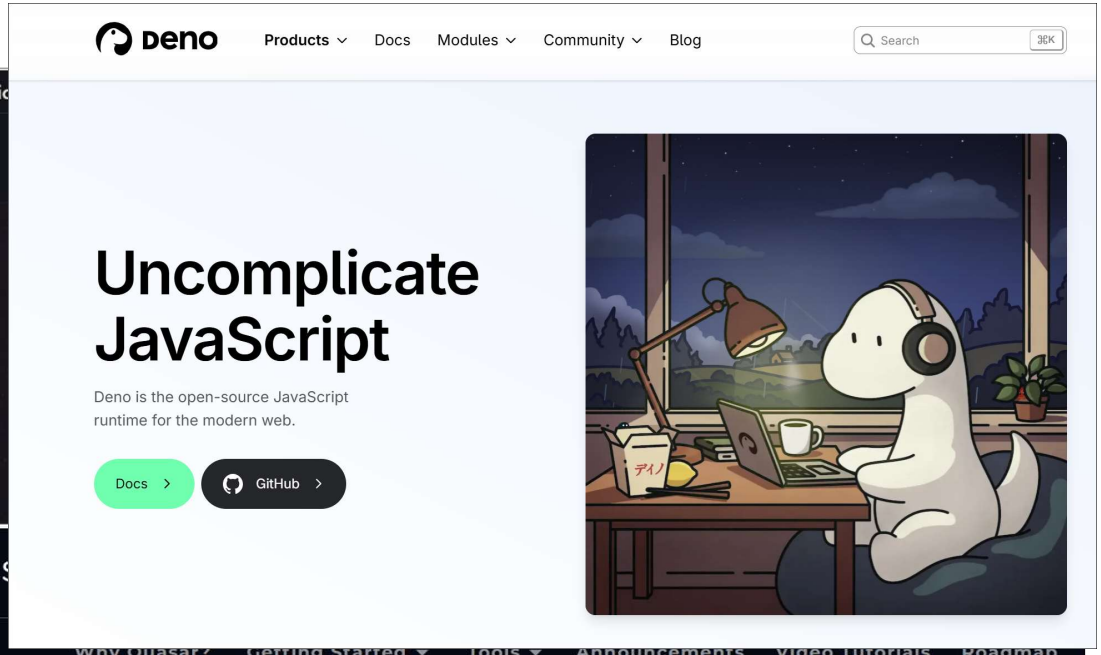
Managed in one central tool

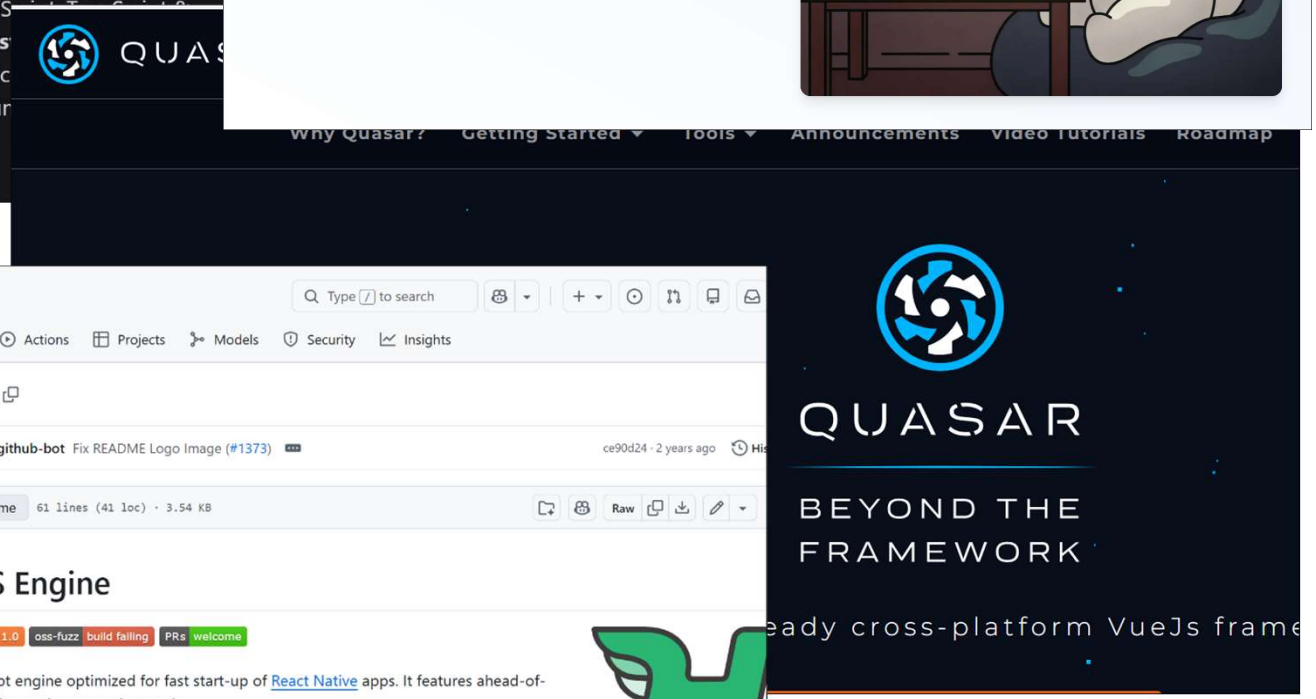


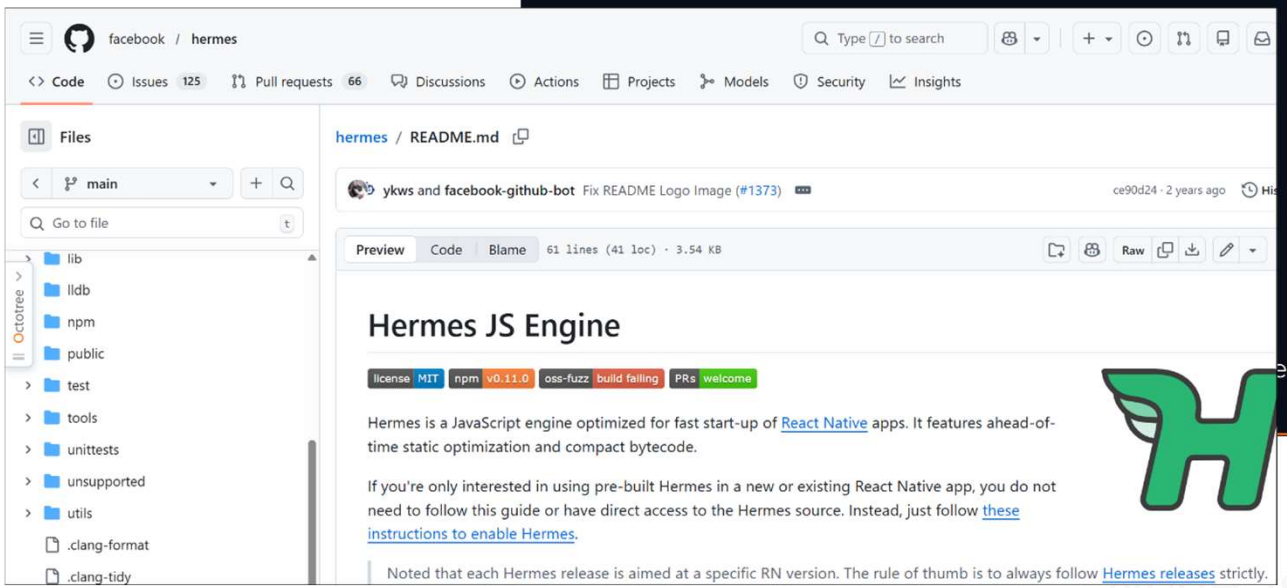
https://nodejs.org/en

Alternatives to NodeJS







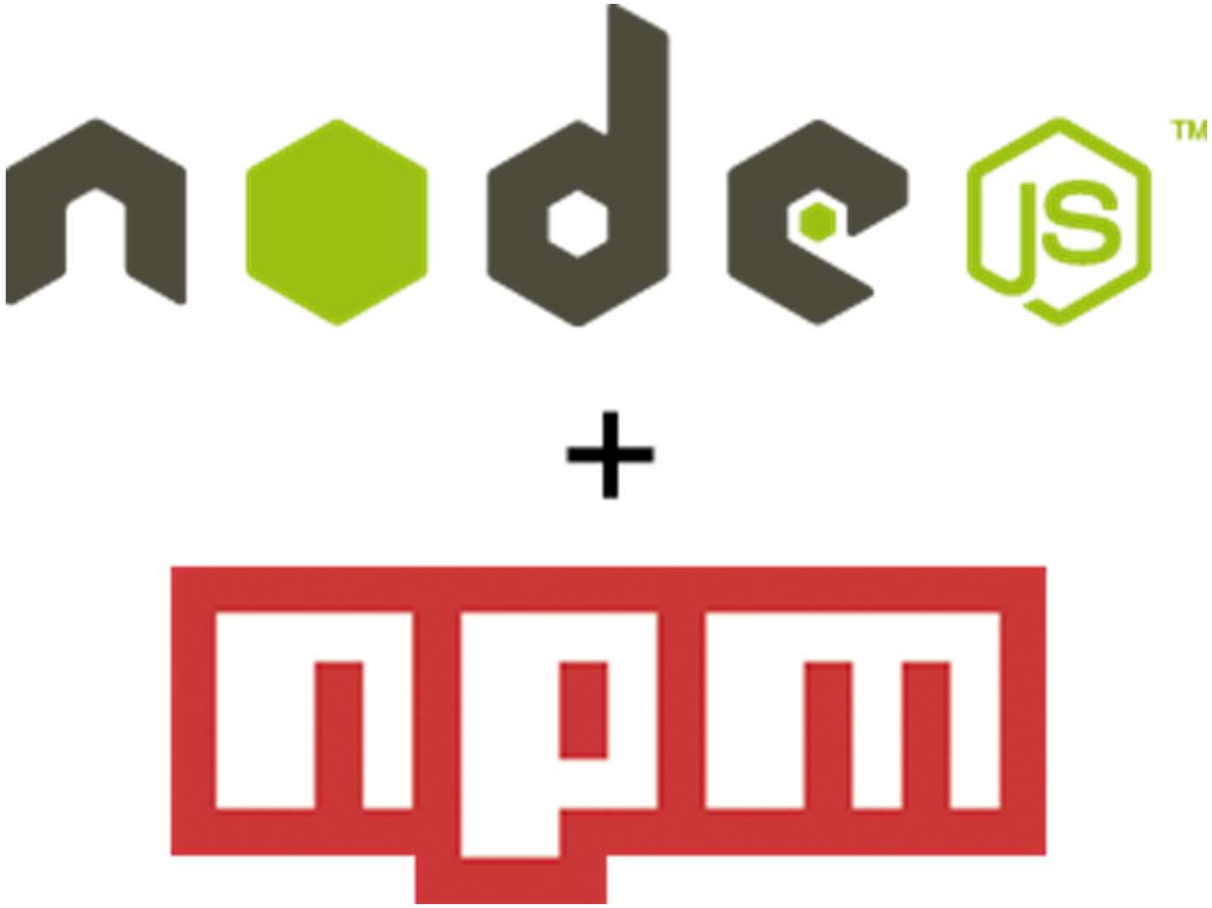


Bottomline – you don't have to learn them, but be aware of their existence. There are node.js alternatives with the same goal

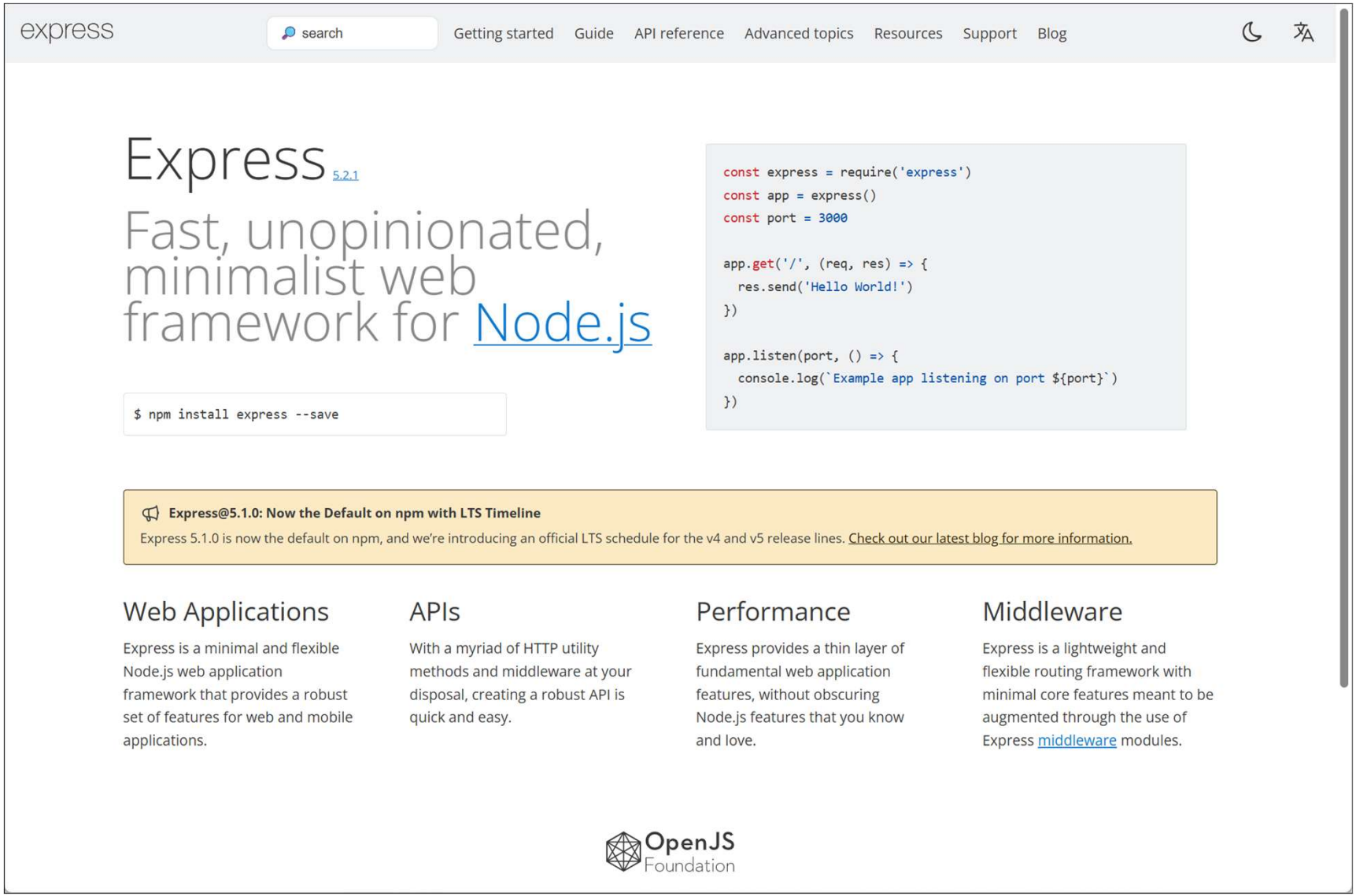
NodeJS : 2 types of Modules

- CommonJS Modules:
  - `myModule.js ➔ Module.exports = myFunction // export`
  - `const myFunction = require('./myModule') // import`
  - Still used A LOT in existing code bases

- ES Modules (modern, but not widespread yet)
  - `myModule.js ➔ export default myFunction = "…." // export`
  - `import myFunction from './myModule') // import`
  - Better treeshaking, more modern notation
  - But: NO NEED to refactor/switch module systems. Node works perfectly fine with both


node
JS
™
+
npm

Express.js framework

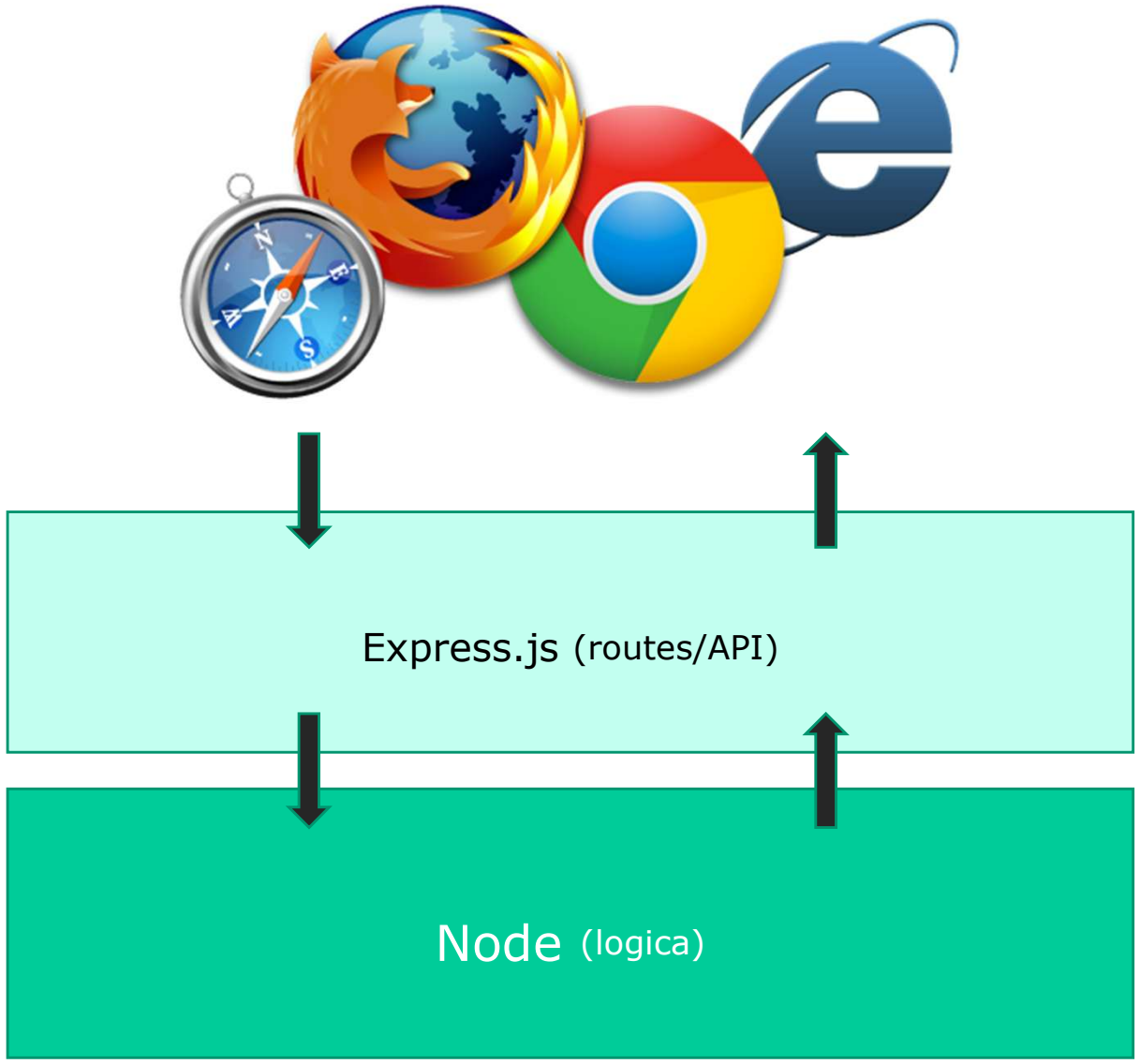


https://expressjs.com/

Express: twee manieren

1. Globally: `npm install -g express-generator`
   - Generator
   - Snelle scaffolding van kale basiswebsite
   - View Engines (Jade, Pug, EJS)
   - Voordeel: snel op weg
   - Nadeel: veel code die je wellicht niet wilt gebruiken, of volgens andere conventies
2. Locally: `npm install express`, per project
   - Creating API's

Middleware



Express API maken

Routes schrijven binnen je app

```
app.get('/api/auteurs', function (req, res) {
   // retourneer gegevens, hier een JSON-bestand met auteurs
   res.json(auteurs);
});
```

- Elke functionaliteit krijgt een eigen RESTful route
- 20 routes? ➔ 20 verschillende endpoints
- Routeparameters: dubbele punt / dynamische routes

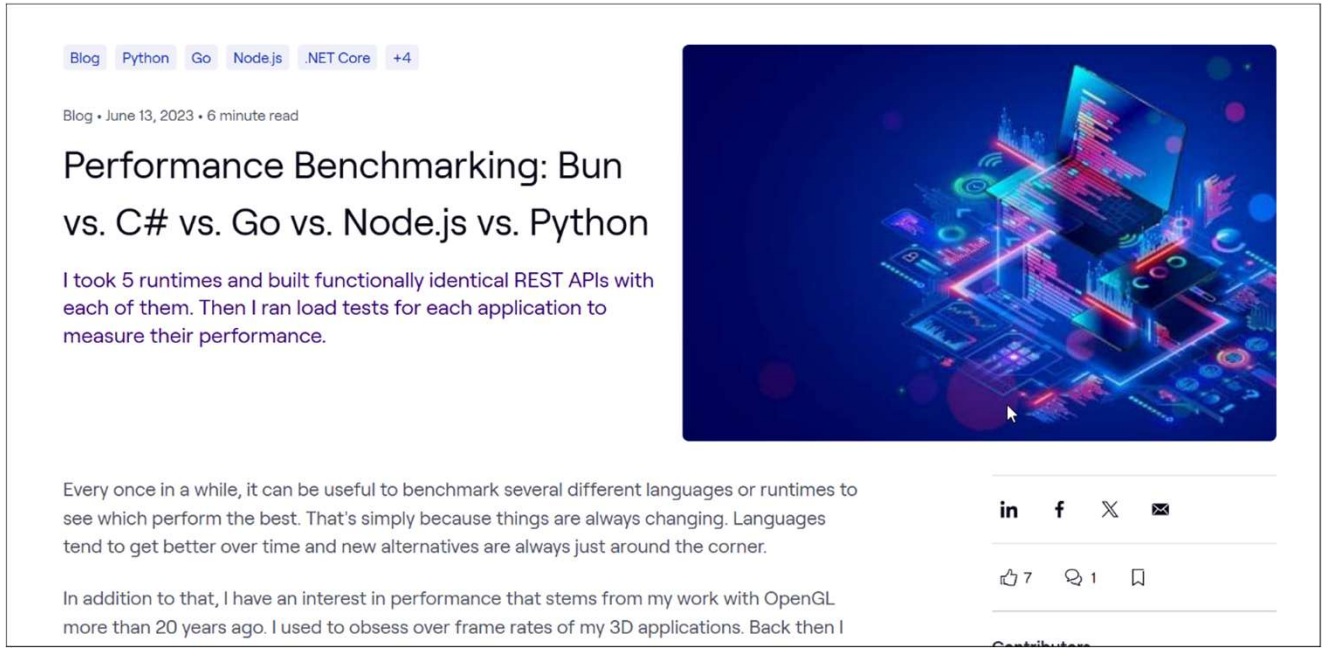
```
app.get('/api/boeken/:id', function (req, res) {…});
```

Questions: Node Performance?

- Overall impressions, common ideas:
- Rust is generally the fastest
  - Rust handles 72.000 req/s vs Node.js 8.000 – 15.000 req/sec, depending on RAM and tasks
- Node.js sits in the middle
  - much faster than Python for I/O-heavy work, slower than Rust/Go for CPU-intensive tasks.
- Python is the slowest for raw computation
  - libraries like NumPy/PyTorch offload to native code, so in practice it's more nuanced.

Some benchmarks:

- https://programming-language-benchmarks.vercel.app/rust-vs-javascript
- https://www.techempower.com/benchmarks/#section=data-r23
- https://www.wwt.com/blog/performance-benchmarking-bun-vs-c-vs-go-vs-nodejs-vs-python



AI on benchmarks:

"Node.js is V8 JIT-compiled so it's surprisingly fast for dynamic code, but it's single-threaded by nature (event loop).

Rust has no GC and compiles to machine code, so it wins on CPU and memory.

Python's GIL is its big bottleneck for parallelism.

The real-world answer always depends on the workload type"

Using multiple import types in a module?

- Not recommended
- Possible in one direction: Importing CommonJS in ESM, NOT the other way around.
- See example `300-mixed-imports`

```
// Modern ESM imports:
import { createRequire } from 'module';
import { add } from './math.mjs';

// Bridge to load old-skool CJS modules:
const require : Require  = createRequire(import.meta.url); // edito
const { greet } = require('./greeter.cjs');

console.log(greet('Peter'));   // Hello, Peter!
console.log(add(2, 3));        // 5
```

Today:

- Finish Express – API's and middleware
- React:
  - Concepts
  - Structure + architecture + building
  - Components (classes + functions)
  - Hooks
  - State
  - Props
  - Data

Tomorrow

- React Data (cont'd)
  - Displaying details
- React Forms
  - user input
- External data / API's
- Evals/goodbye